WEEK 4 SPECIALIST ADVANCED BUILD | VIDEO WALKTHROUGH SCRIPT | PRESENTER: AHMED (MEMBER 6)

Video Walkthrough & Speaking Script

Complete Minute-by-Minute Screen & Speaking Roadmap (5 to 8 Minutes)

PRESENTER NAME	TRACK & INTERNSHIP	PROJECT NAME	RECOMMENDED TIME
Ahmed (Member 6)	Web Development Specialist (SAFEX)	DevPulse Public API Product	5 – 7 Minutes

📋 Video Record Karne Se Pehle Yeh 4 Cheezein Open Kar Lein:

✅ **Tab 1 (Browser):** `http://localhost:3000/docs` (Swagger UI)

✅ **Tab 2 (Browser):** `http://localhost:3000` (Developer Portal)

✅ **VS Code:** `README.md` aur `openapi.yaml` open rakhein

✅ **Terminal 1:** `npm start` chal raha ho (port 3000 par)

1. Introduction & Business Context

0:00 – 1:15

🖥️ **Screen Par Kya Dikhana Hai:** VS Code mein **README.md** khol kar title aur executive summary dikhayein.

English: "Hello everyone and respected evaluators. My name is Ahmed, Member 6 in the Web Development track of the SDC Internship. Today I am presenting my Week 4 Specialist Advanced Build deliverable: **DevPulse Public API Product**.

Moving beyond Week 3's frontend e-commerce storefront, Week 4 required a completely different problem and skill set: **Backend API engineering, contract-first design with OpenAPI 3.0, Express.js middleware architecture, multi-tenant rate limiting, and RFC-compliant versioning**.

DevPulse Labs is an independent software house that specializes in static code analysis and engineering metrics. My role was to package their internal analytics engine into a commercial public API product with tiered API keys, fair-use rate limits, and interactive documentation."

URDU VERSION (AGAR URDU/ENGLISH MIX BOLNA HO):

"Mera naam Ahmed hai, aur main Week 4 ka project DevPulse Public API Product present kar raha hoon. Week 3 frontend e-commerce tha, jabke Week 4 mein humne backend API engineering, OpenAPI 3.0 contract-first design, tiered API keys aur rate limiting implement ki hai DevPulse Labs ke liye."

2. OpenAPI 3.0 Contract & Live Swagger UI Demo

1:15 – 2:45

> 🖥️ **Screen Par Kya Dikhana Hai:** Browser switch karein `http://localhost:3000/docs` par. Green **Authorize** button click karein, `devpulse_pro_demo_key` enter karein, phir `GET /metrics/code-health` execute karke dikhayein.

English: "Following professional software standards, I followed a **contract-first approach**: before writing backend logic, I authored a comprehensive OpenAPI 3.0 specification in `openapi.yaml`.

Here in the browser, you can see our live Swagger UI portal at `/docs`. It documents the 5 core endpoints: `/metrics/code-health`, `/metrics/security-audit`, `/metrics/tech-debt`, `/metrics/build-benchmarks`, and `/analyze/repository`.

Let's authenticate: I click 'Authorize' and enter our Developer Tier key: `devpulse_pro_demo_key`. Now when I execute `GET /metrics/code-health`, we get an immediate **200 OK** with our standardized response envelope, and in the response headers, we see our rate limit headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset`."

3. Special Constraint: Comparison Benchmark Proof

2:45 – 4:30

🖥️ **Screen Par Kya Dikhana Hai:** VS Code ke Terminal par switch karein aur yeh command run karein:

```
npm run test:benchmark
```

English: "The most critical requirement of Week 4 is the **Special Constraint: Comparison Rule**, where we must evaluate two alternative approaches to the hardest part of the task.

For rate limiting, I implemented and micro-benchmarked **Fixed Window Counter** versus **Sliding Window Counter**.

As shown in this benchmark output, the Fixed Window algorithm suffers from a dangerous **boundary burst vulnerability**: if a client sends 20 requests at the 59th second and 20 requests at the 61st second, the Fixed Window allows **40 requests in 2 seconds (a 200% capacity spike)**.

In contrast, our **Sliding Window Counter** uses a rolling weighted average. It immediately blocks 19 of the 20 requests at the window boundary, strictly capping the burst to 21 requests. Since both algorithms operate at sub-microsecond latencies (> 1.7 million operations per second), I justified and selected **Sliding Window** as the production standard."

4. Burst Load Test (429 Too Many Requests Proof)

4:30 – 5:45

🖥️ **Screen Par Kya Dikhana Hai:** Terminal mein yeh command run karein aur 200 → 429 transition dikhayein:

```
npm run test:ratelimit
```

English: "Now let's prove the rate limits under automated burst traffic using `npm run test:ratelimit`.

On the Free Tier (quota of 5 requests per minute):
- Requests 1 through 5 succeed with **HTTP 200 OK**, and the remaining counter drops from 4 down to 0.
- On request 6, the system immediately throttles with **HTTP 429 Too Many Requests** and attaches an RFC-compliant `Retry-After: 5s` header.

This empirically proves that fair-use quotas and automatic client throttling work flawlessly without server crashes."

5. Versioning, Developer Portal & Conclusion

5:45 – 7:00

🖥️ **Screen Par Kya Dikhana Hai:** Browser mein `http://localhost:3000` (Developer Portal) dikhayein, phir terminal mein master test run karein: `node scripts/run-all-tests.js`

English: "For API versioning, we operate dual-track endpoints:
- `/api/v1` is deprecated with RFC 8594 standard headers: `Deprecation: true`, `Sunset` date, and successor `Link` headers.
- `/api/v2` is our active production version with camelCase properties and standardized response envelopes.

Here in the browser is our interactive **Developer Portal** where users can provision API keys and monitor real-time telemetry.

To conclude, running our master test suite shows all **6 test suites passing with 100% success**. All code is pushed to GitHub at `github.com/axediqbal/safexWeek4`. Thank you very much!"

💡 Evaluator Marks Ke Liye 3 Golden Rules:

- **Confidence:** Aaram se aur normal speed mein bolein. Agar thodi bohot ghalti ho jaye to rukna nahi, agla point bolte jayein.
- **Emphasize Special Constraint:** Benchmark table dikhate waqt batayein ke "Fixed Window allows 200% burst, so I selected Sliding Window". Evaluators is par extra marks dete hain!
- **Show Live Terminal:** Commands live run karke dikhane se evaluator ko yakeen ho jata hai ke project 100% working hai!